StudyForge AI

30 Page Beginner Project Book

Theory → Code → PPT → Viva, explained in simple language

Use one page at a time. Understand before you memorise.

Page 2 How to Study This Book

This book is written for a beginner. Do not rush through it as if it were a report. Read one page, say the idea aloud in your own words, then open the project file named on that page. If a term is unfamiliar, learn the simple meaning first. The goal is understanding, not copying difficult vocabulary.

When	What to do
First reading	Read pages 3 to 18 for theory and project flow.
Second reading	Open the related code files and trace one feature.
PPT practice	Use pages 21 to 24 to build slides and demo.
Viva practice	Answer pages 25 to 29 without looking at the answers first.

- Keep the running project open when possible.
- Write a one-line meaning for every new term.
- Do not claim technology that the code does not use.
- When stuck, return to the student journey: login, upload, ask, practise.

Check yourself: What should be your first goal? Explain the student journey in simple words before explaining code.

Page 3 The Project Story

StudyForge AI is a study workspace for BTech students. It turns a student's own PDF into useful study activities. A student can create an account, upload study material, ask ASHIEY questions, make flashcards, attempt quizzes, track revision topics, maintain tasks and personalise a profile.

The project solves a real student problem. Notes, PDFs, doubts and revision plans are normally disconnected. A student reads a PDF in one app, searches for explanations in another, writes tasks elsewhere and may never test recall. StudyForge brings these jobs together.

Student need	StudyForge feature	Learning benefit
Understand notes	Ask ASHIEY chat	Explanation linked to material
Revise quickly	Flashcards and recall	Active recall
Test knowledge	Quiz and score	Find weak areas
Plan work	Planner and revision tracker	Organised study
Feel ownership	Profile, theme and avatar	Personal study space

Check yourself: Give the project definition in one line. StudyForge AI is a full-stack study workspace that turns student PDFs into source-based chat and revision tools.

Page 4 What is a Web Application

A web application is a program that runs through a browser. It is not only a static webpage. It can accept user input, save records, call a server and change its display. StudyForge is a web application because students sign in, upload files, save tasks and receive personalised results.

Most web applications have a client and a server. The client is the browser-side application used by the student. The server is the trusted program that receives requests and performs work. In StudyForge, React runs in the browser and FastAPI runs on the server.

```
Student browser = client
React / Next.js interface = client application
FastAPI = server application
SQLite = server-side data storage
Gemini API = external AI service
```

Restaurant analogy: the student is a customer, the browser is the waiter, the FastAPI backend is the kitchen and SQLite is the store room. The customer should not go directly to the kitchen or store room. That is why the browser does not access the database or AI key directly.

Check yourself: Why is the backend called trusted? Because it controls private data, validates actions and keeps secret API keys away from the browser.

Page 5 API and HTTP Basics

An API is a set of rules that lets two programs communicate. In this project, the frontend sends an HTTP request to an API endpoint and FastAPI sends a response. An endpoint is one URL plus one method. For example, POST /api/documents/upload means create an upload request at that URL.

HTTP method	Meaning	StudyForge example
GET	Read information	GET /api/documents
POST	Create or trigger work	POST /api/quizzes/generate
PUT	Update information	PUT /api/profile
DELETE	Remove information	Delete a task or conversation

JSON is the text format used for structured request and response data. A quiz request may send document ID and count. The response may send quiz ID and question list. JSON is easy for React and Python to read.

```
Request body:
{ "document_id": "abc123", "count": 5 }

Response:
{ "quiz_id": "quiz456", "questions": [ ... ] }
```

Check yourself: What is an endpoint? A specific URL and HTTP method that represents one backend action.

Page 6 Frontend Theory

The frontend is everything the student sees and clicks. StudyForge uses Next.js, React and TypeScript. Next.js gives project structure and build tools. React builds an interface from components. TypeScript adds type checking to JavaScript so data mistakes can be found earlier.

Word	Beginner meaning	Project example
Component	Reusable UI piece	Planner or AshieyChat
Props	Information given to a component	token, user, onSaved
State	Information that can change	selected PDF, quiz, busy
Event	User action	click Create quiz
Render	React draws current UI	show login or workspace
CSS	Visual styling	colors, spacing, animation

React is useful because a page changes many times. Before sign-in, the page shows login. After sign-in, it shows the workspace. After quiz generation, it shows question buttons. React changes the display from state instead of forcing the developer to manually edit HTML.

Check yourself: What is the main job of frontend? Show the interface, receive user input and call backend APIs.

Page 7 React State and Effects

State is data that can change while a React page is open. StudyForge page.tsx uses state for token, user, current page, selected document, cards, quiz, notices and loading state. When state changes, React renders the correct screen again.

```
const [quiz, setQuiz] = useState([])
const [documentId, setDocumentId] = useState("")
const [busy, setBusy] = useState(false)
```

useState gives two things: the current value and a setter function. setQuiz changes quiz. A component should use a setter rather than changing the variable directly. This helps React know when to update the screen.

useEffect is used for side effects: jobs outside normal drawing. StudyForge uses it to restore a saved token from local storage and to load documents when the user opens flashcards, quizzes or games.

- State: data used to draw screen.
- Setter: safe way to change state.
- Effect: job such as fetch data after render.
- Local storage: browser storage for saved token.

Check yourself: What happens after setQuiz receives questions? React renders again and QuizView can display the new questions.

Page 8 Important Frontend Files

File	Responsibility
frontend/app/layout.tsx	Loads CSS, theme tools, motion control and page metadata.
frontend/app/page.tsx	Coordinates login, navigation, selected PDF, quiz and cards.
frontend/app/ashiey-chat.tsx	Shows chat input, streaming answer and saved conversation features.
frontend/app/library.tsx	Organises subjects and uploaded materials.
frontend/app/planner.tsx	Creates and manages student tasks.
frontend/app/revision.tsx	Tracks learning, revision and confident topic status.
frontend/app/profile-setup.tsx	Saves student profile preferences.
frontend/app/avatar-creator-3d.tsx	Lets user customise avatar look.
frontend/app/atelier.css and workspace.css	Give the warm workspace visual design.

When an examiner asks where a feature is implemented, do not say everything is in page.tsx. page.tsx coordinates the screen, but specialised components own their feature interface. This separation makes code easier to understand and maintain.

Check yourself: Where would you look for the chat UI? frontend/app/ashiey-chat.tsx.

Page 9 Backend Theory

The backend is the trusted Python program. StudyForge uses FastAPI. FastAPI lets a developer write Python functions and attach them to web URLs. It also validates data and creates automatic API documentation at /docs.

```
@app.get("/health")
def health() -> dict:
    return { "status": "healthy", "project": "StudyForge AI" }
```

The line starting with @ is a decorator. It tells FastAPI that a browser request to /health should run the health function. The function returns a Python dictionary. FastAPI turns that dictionary into JSON.

backend/app/main.py contains important routes for accounts, PDF upload, chat, flashcards, quizzes, profile and progress. backend/app/study.py adds routes for library, planner, revision and saved conversations.

Check yourself: What is a route? A backend function connected to an API URL and HTTP method.

Page 10 Validation and Pydantic

Validation means checking whether incoming data is safe and expected before using it. FastAPI uses Pydantic models. A model describes which fields should exist and what type each field has. This is better than trusting any JSON sent by a browser.

Model	Purpose	Example data
SignupRequest	Account creation	name, email, password
ChatRequest	Ask a study question	question, document ID, mode
GenerateRequest	Create cards or quiz	document ID, count
QuizSubmission	Submit answers	answer dictionary
AvatarRequest	Save customisation	avatar object, gender

GenerateRequest restricts count from 1 to 20. Without this rule, someone could send zero, a negative number or an extremely large number. Validation saves later functions from dealing with bad values.

FastAPI uses HTTPException for clear errors. For example, 401 means login is required. 413 means uploaded file is too large. 422 means request format may be valid but the project cannot make a useful result, such as a distinct quiz from too little material.

Check yourself: Why validate before processing? It prevents invalid values from reaching database, PDF and AI logic.

Page 11 Database Theory

A database stores organised data after a page refresh or server restart. SQLite is a relational database stored in one local file. It is a good choice for a BTech project because it needs no separate database server.

Term	Easy meaning	StudyForge example
Table	Category of records	users
Row	One stored item	one student
Column	One property	email
Primary key	Unique ID	user ID
Relationship	Connection between records	documents.user_id
CRUD	Create, Read, Update, Delete	signup, list, edit, remove

A relation explains how tables connect. One user can own many documents. Therefore documents stores user_id. When the backend receives a document ID, it checks that user ID before it gives access. This is a basic but important privacy rule.

```
users:        id | name | email | password_hash
documents:    id | filename | user_id
chunks:       id | document_id | page | text
```

Check yourself: Why use a primary key? It gives every row a unique identity so the backend can find the correct record.

Page 12 StudyForge Database Tables

Table	Stores	Why
users	Account, preferences, avatar, subjects	Personal student workspace
documents	PDF name and owner	Know which student owns material
chunks	Extracted text and page	Searchable source evidence
conversations	Chat messages and mode	Continue previous study chats
subjects and document_subjects	Library structure	Organise notes
student_tasks	Planner title, date, state	Manage study work
revision_topics	Topic and learning status	Track revision
flashcards	Question and answer	Store practice set
quizzes	Full question set	Protect answer before submit
quiz_attempts	Score and total	Calculate progress

The database is created or upgraded in initialize_database. The code creates tables if they do not exist. It also adds newer columns such as branch, semester, college, avatar and gender if an older local database is already present. This is a simple migration approach for project development.

Check yourself: What is a migration? A controlled database structure update, such as adding a new column to an existing table.

Page 13 Authentication and Authorization

Authentication asks: who are you? Authorization asks: are you allowed to do this? These are different. In StudyForge, login authenticates a student. owned_document authorizes whether that student may use a chosen PDF.

Passwords are not saved as normal readable text. The code uses PBKDF2 HMAC SHA 256 with a random salt and 120000 iterations. Hashing is one-way: the backend compares a new hash rather than decrypting a stored password.

1. Student signs up with password.
2. Backend creates random salt and password hash.
3. Backend stores only hash and salt-derived value.
4. Student logs in.
5. Backend compares computed hash with stored hash.
6. Backend creates signed token if match.

The frontend stores the token locally and sends Authorization: Bearer token on protected requests. current_user verifies token signature and expiry, then loads the student record. This project uses a custom base64 plus HMAC token, not a JWT library.

Check yourself: Give an example of authorization. Before generating a quiz, the backend checks the selected document belongs to the logged-in student.

Page 14 PDF Upload Theory

A PDF upload must be treated carefully. The backend checks file type, maximum size, page count and encryption. StudyForge accepts application/pdf, reads up to 20 MB, accepts 1 to 200 pages and rejects password-protected PDFs.

PyMuPDF opens the PDF and reads text from each page. The backend saves the original PDF under a generated document ID and also saves extracted text chunks in SQLite. Keeping both is useful: chunks are fast for retrieval, while original PDF can be used by a configured AI provider.

Upload step	Reason
Check login	Only signed-in students upload.
Check PDF rules	Avoid unsupported or dangerous input.
Extract page text	Create searchable study content.
Split into chunks	Make retrieval focused.
Save PDF file	Keep original source.
Save document and chunks	Link material to student and pages.

A scanned PDF may be a collection of images rather than real text. The current local extraction can be weak for such files. The project notes this limitation honestly. OCR is a future improvement.

Check yourself: What happens if PDF is 25 MB? Backend returns a file-too-large error instead of processing it.

Page 15 Chunking Theory

A full textbook chapter is too long and broad to search as one item. A chunk is a small portion of text. StudyForge splits text into around 180 words with 30 words overlap. Overlap repeats the last words of one chunk at the start of the next.

```
Chunk 1: words 1 to 180
Chunk 2: words 151 to 330
Shared overlap: words 151 to 180
```

Why not split every 180 words with no overlap? Imagine a definition starts at word 170 and finishes at word 190. Without overlap, neither chunk contains the whole thought. With overlap, chunk 2 contains the end of the first chunk and the complete definition context is more likely to appear.

Each chunk stores document_id, page and text. The page field is important because students can open the original material to verify an answer. Source page makes the study assistant more transparent.

Check yourself: What is chunk overlap? A small repeated region between nearby chunks that protects context at boundaries.

Page 16 RAG Theory

RAG means Retrieval Augmented Generation. Retrieval means finding relevant source chunks. Generation means writing a natural answer. The word augmented means the model gets extra relevant material before answering.

StudyForge retrieval uses TF IDF and cosine similarity. It loads chunks from the selected document, converts chunks and student question to weighted word vectors, calculates similarity and selects the top relevant chunks with positive scores.

Step	Example for question What is a process?
1	Load chunks from selected operating systems PDF.
2	Convert chunks and question into TF IDF vectors.
3	Score similarity for each chunk.
4	Choose chunk containing program in execution.
5	Use it as source context for answer.

RAG does not guarantee truth. It makes answers more connected to the uploaded material. Quality depends on readable PDF text, correct retrieval and responsible provider response.

Check yourself: What is the difference between RAG and a normal generic chatbot? RAG first finds relevant material from the selected PDF before generating an answer.

Page 17 TF IDF and Cosine Similarity

TF means term frequency. A word used many times in one chunk gets more importance there. IDF means inverse document frequency. A word that appears in every chunk gets less importance because it does not distinguish one topic from another.

For example, in an operating systems PDF, the word system may appear everywhere and carry little special meaning. The word deadlock may appear only in a specific explanation, so it becomes useful for finding the deadlock chunk.

Cosine similarity compares two vectors. You do not need to calculate it manually in viva. Say: it gives a score for how similar the important weighted words in the question are to each text chunk. Higher score means more likely relevance.

```
question: "What is a page fault?"
high score chunk: "A page fault occurs when..."
low score chunk: "TCP uses a three-way handshake..."
```

Check yourself: Why is TF IDF useful? It helps give more weight to topic-specific words than common filler words.

Check yourself: Why is cosine similarity useful? It ranks chunks so the backend can choose relevant evidence.

Page 18 Generative AI and Providers

A large language model, or LLM, generates useful text from input. StudyForge uses Google Gemini for document-aware answers and study item generation when a key is configured. It also includes an optional Groq path for streamed chat.

StudyForge does not train a new model. It calls an external API. The project controls what it sends: the question, selected study mode, recent conversation history, retrieved source excerpts and, where configured, the original PDF.

Input to AI	Why
Student question	Tells model what to explain.
Study mode	Requests simple explanation, hint, code review or viva practice.
History	Lets follow-up questions make sense.
Retrieved excerpts	Grounds response in source.
PDF	Lets provider inspect full selected material.

Keys such as GEMINI_API_KEY remain in backend environment variables. They are never placed in frontend code or Git. This is essential because browser code can be inspected by anyone.

Check yourself: Did you train Gemini? No. The project uses Gemini through its API and controls context, validation and display.

Page 19 Streaming and Fallbacks

Streaming sends a response in small pieces rather than waiting for one complete long answer. StudyForge chat stream sends a status event such as Reading your PDF, then delta events with new text, and finally a done event with final result and sources.

```
status → "Reading your PDF"
delta  → "A process is..."
delta  → "a program in execution..."
done   → answer plus sources
```

This improves experience because the student receives feedback immediately. It also lets the interface handle a long answer without appearing frozen.

External providers can fail because of rate limits, network issues or temporary busy status. StudyForge has fallback_answer for retrieved source text. It also has a local quiz fallback. Good engineering means a provider error does not automatically make the whole study tool useless.

Check yourself: What is a fallback? A useful alternative behaviour used when the preferred provider or service is unavailable.

Page 20 Flashcards and Active Recall

Active recall means trying to remember before seeing an answer. Flashcards support this. In StudyForge, the student chooses a PDF, creates cards, sees a concept label and question, then presses Reveal answer. They can move previous, next or shuffle.

The backend tries Gemini generation when available. It asks for different concepts, validates answers and source pages, and filters close duplicates. If generation is unavailable, the project can form basic cards from clear sentences in stored chunks.

Flashcard field	Meaning
question	The prompt shown first.
answer	Explanation shown after Reveal.
focus	Short label such as Virtual Memory.
source_page	Where the material came from.

The most important learning design decision is not to show all answers at once. The student should pause and attempt recall. This is more useful for revision than rereading a paragraph.

Check yourself: Why use a focus label? It tells the student which concept the card is practising and makes the set feel organised.

Page 21 Quiz Theory

A quiz is more structured than flashcards. Each question has four options, one correct answer, an explanation and a source page. The backend stores complete questions including answers. The initial frontend response removes answer and explanation so students cannot see them before submitting.

7. Student selects PDF and presses Create quiz.
8. Frontend sends document ID and requested count.
9. Backend checks document ownership.
10. Backend generates and validates candidate questions.
11. Duplicate filter keeps different concepts.
12. Backend stores full quiz in SQLite.
13. Frontend shows only safe question fields.
14. Student submits choices and backend calculates result.

When a student submits, the backend checks every question has an answer, compares choices with stored correct options, stores the first attempt and calculates XP. Each correct answer earns 10 XP. The frontend then shows score, correct answer, explanation and source page.

Check yourself: Why are correct answers stored on server? To preserve quiz integrity. Sending them at creation would make it easy to inspect answers before attempting.

Page 22 Duplicate Filtering and Local Quiz Fallback

AI can create two questions that look different but test the same idea. For example, What is virtual memory and Explain virtual memory are repetitive. StudyForge creates extra candidate items, then removes common stop words, compares meaningful overlap and uses sequence similarity to identify close paraphrases.

The functions study_words, study_text_is_similar, focus_from_question and keep_distinct_study_items work together. The system would rather return fewer useful questions than fill a quiz with the same concept repeated five times.

If Gemini returns a temporary busy result, make_local_quiz uses sentences already extracted from the PDF. It chooses a source sentence as correct answer and sentences from other topics as distractors. This fallback is more literal but it remains useful, source-grounded and available.

Check yourself: Why is local fallback better than only an error message? A student can continue practising even when an external service is temporarily unavailable.

Page 23 Avatar Theme and Accessibility

StudyForge has a warm creative studio visual identity. The interface uses ivory, lavender and ink-like colors, soft animated details and handwritten accents. The goal is to make difficult engineering study feel calmer and more personal.

The avatar creator supports gender-aware choices and customisation. The data is saved through a protected profile API and stored as JSON in the user record. The avatar has a 3D-styled appearance made with layered React and CSS.

Be exact in viva: this is not a heavy WebGL or Three.js scene. It is a lightweight 3D-styled interface. That was a practical design choice because it gives a personal appearance without adding complex graphics dependencies.

Accessibility choice	Why
Theme toggle	Supports comfortable light or dark view.
Reduced-motion support	Does not force animation on every student.
Clear labels	Actions such as Create quiz and Reveal answer are understandable.
Text-safe rendering	Chat avoids blindly trusting raw HTML.

Check yourself: Why respect reduced motion? Some users find animation distracting or uncomfortable, so the interface should allow calmer interaction.

Page 24 Run Test and Debug

The simplest Windows command is START-STUDYFORGE.cmd. It starts backend and frontend, waits for them to respond, and opens the browser. The backend normally uses port 8000. The frontend uses port 3000.

```
cd "C:\Users\akshita sharda\Documents\ChatGPT\STUDYFORGE_AI"
.\START-STUDYFORGE.cmd
```

Problem	Likely reason	What to do
Port already used	Old project instance is still running	Use STOP-STUDYFORGE.cmd then start again.
Frontend page opens but API fails	Backend is not healthy	Open http://127.0.0.1:8000/health.
Upload fails	Wrong PDF rule	Check type, size, pages and encryption.
Quiz has no questions	PDF has little readable distinct text	Use clearer text-based PDF.
Provider busy	External service issue	Try again; local quiz fallback should help.
Login expired	Saved token timed out	Sign in again.

Testing checks that code behaves as expected. Backend uses pytest. Frontend uses a production build and browser check scripts. A test is not proof that everything is perfect, but it catches known behaviours after changes.

Check yourself: What does health endpoint prove? It proves the backend server is running and responding to HTTP requests.

Page 25 PPT Plan

Your PPT should tell a story, not show every code file. Keep slides visual. Use short points, screenshots of your running project and one architecture diagram. Explain details verbally from this book.

Slide	Main message
1	StudyForge AI and team introduction.
2	Problem: BTech PDFs and revision work are scattered.
3	Solution: one personalised study workspace.
4	Features: upload, chat, cards, quiz, planner, revision and avatar.
5	Architecture: React → FastAPI → SQLite, PDF, retrieval and Gemini.
6	PDF and RAG: extract, chunk, retrieve, generate.
7	Backend: validation, tokens, ownership and database.
8	Frontend: workspace UI, state, theme and accessibility.
9	Reliability: distinct questions and local fallback.
10	Testing, limitations and future scope.

Do not promise that AI is always correct. Say it uses retrieved material, source pages and fallbacks to improve usefulness. This sounds mature and accurate.

Check yourself: What is the best architecture slide sentence? The React frontend calls FastAPI, which validates requests, stores records in SQLite, extracts PDF text, retrieves relevant chunks and optionally calls Gemini.

Page 26 Live Demo Script

Prepare one short readable PDF before the viva. A stable demo is better than a large complicated PDF. Keep the application already running. Explain each action while doing it; do not click silently.

15. Open StudyForge and sign in.
16. Show home workspace, profile or avatar.
17. Open Subject Library and upload a short PDF.
18. Ask ASHIEY one direct question from PDF.
19. Point out answer source page if shown.
20. Open Flashcards, identify the focus label and reveal answer.
21. Open Quiz, choose answers and submit.
22. Show score or progress.
23. Open Planner or Revision Tracker to show this is a full workspace.

If Gemini is slow, do not panic. Say: The system has provider-dependent generation, but the backend also retrieves local document chunks and includes a local quiz fallback. Then move to another feature or explain the architecture.

Check yourself: Why should you use a short PDF for demo? It uploads and extracts faster and makes source-grounded results easier to explain.

Page 27 Basic Viva Questions

Check yourself: What is full stack? A project with both frontend interface and backend server logic.

Check yourself: What is React? A JavaScript library used to build interface from reusable components.

Check yourself: What is FastAPI? A Python framework used to create validated backend APIs.

Check yourself: What is an API? A communication contract between programs using endpoints, methods and data.

Check yourself: What is JSON? A structured text format used in web requests and responses.

Check yourself: What is a database? Persistent organised storage for application records.

Check yourself: Why SQLite? It is simple and local, suitable for a college prototype.

Check yourself: What is a table? A collection of similar database records.

Check yourself: What is authentication? Proving which user is making a request.

Check yourself: What is authorization? Checking whether that user has permission for a resource.

Check yourself: Why hash password? To avoid storing readable passwords.

Check yourself: What is a token? A signed value that proves a logged-in session.

Check yourself: What is a component? A reusable React UI part.

Check yourself: What is state? Data that can change and cause React to re-render.

Check yourself: What is CSS? A language for visual styling of HTML elements.

Page 28 RAG AI and Study Tool Viva Questions

Check yourself: What is RAG? Retrieval Augmented Generation: retrieve relevant source text before generation.

Check yourself: Why use RAG? It keeps answers closer to the student PDF and supports source pages.

Check yourself: What is TF IDF? A word weighting method that gives topic-specific words more importance.

Check yourself: What is cosine similarity? A score for similarity between question and chunk vectors.

Check yourself: What is chunking? Dividing a large PDF text into smaller searchable pieces.

Check yourself: Why chunk overlap? It preserves context at a boundary between chunks.

Check yourself: What is an LLM? A large language model that generates language from input.

Check yourself: Did you train the model? No. The project calls external Gemini API.

Check yourself: What is streaming? Sending answer parts gradually instead of waiting for full answer.

Check yourself: What is fallback? Alternative useful behaviour if preferred service fails.

Check yourself: How do cards help learning? They support active recall before student sees answer.

Check yourself: How do you avoid repetitive quiz questions? Generate extra candidates and filter similarity.

Check yourself: Why hide answers before submit? To stop answers being visible before quiz attempt.

Check yourself: What does XP do? It gives simple visible progress based on correct answers.

Check yourself: What happens if Gemini is busy? The project can show local retrieved material and use local quiz fallback.

Page 29 Advanced Viva Questions

Check yourself: Why is API key not in frontend? Frontend code can be inspected in browser; secrets must remain on backend.

Check yourself: Why use Pydantic? It validates request fields and types before backend logic runs.

Check yourself: What is dependency injection? FastAPI Depends can run shared authentication and give verified user to route.

Check yourself: Why check document owner? A guessed document ID must not allow another student to read private material.

Check yourself: What does 401 mean? Authentication failed or login is missing.

Check yourself: What does 422 mean? Request data is acceptable format but backend cannot create a valid result.

Check yourself: What does 503 mean? A service, often external provider, is temporarily unavailable.

Check yourself: Why store original PDF and chunks? Original keeps source file; chunks make local retrieval fast.

Check yourself: What are current limitations? Weak scanned text, lexical TF IDF, provider availability and SQLite local scale.

Check yourself: Why is TF IDF not perfect? It relies on words, so it may miss the same idea expressed using different language.

Check yourself: What is future scope? OCR, embeddings, vector database, PostgreSQL, standard sessions, caching, rate limits and user evaluation.

Check yourself: Is the avatar WebGL? No. It is a lightweight 3D-styled React and CSS visual.

Check yourself: Why test code? Tests catch behaviour regressions after changes.

Check yourself: What would change for production? Use stronger deployment database, standard auth, monitoring, backups and secure hosting.

Page 30 Final Revision and Speaking Checklist

The day before your viva, do not read the entire codebase again. Revise the project story, architecture, PDF and RAG flow, authentication and quiz fallback. Then rehearse your PPT and demo. A clear simple explanation is more valuable than a difficult memorised answer.

- I can give a one-minute project explanation.
- I can draw React → FastAPI → SQLite → PDF retrieval → Gemini flow.
- I can explain client, server, API and JSON.
- I can explain component, state and useEffect.
- I can explain route, validation and Pydantic.
- I can explain table, user ID and document ownership.
- I can explain password hash and signed token.
- I can explain PDF extraction, chunking and page source.
- I can explain RAG, TF IDF and cosine similarity.
- I can explain provider streaming and fallback.
- I can explain flashcards, quiz answer protection and duplicate filtering.
- I can state limits and future scope honestly.

One Minute Final Answer

StudyForge AI is a full-stack study workspace for BTech students. The React and Next.js frontend gives students a personalised interface for notes, tasks, revision, flashcards and quizzes. The FastAPI backend validates requests, authenticates students and stores data in SQLite. When a student uploads a PDF, PyMuPDF extracts text, the backend creates page-linked chunks and TF IDF with cosine similarity retrieves relevant chunks for study questions. Gemini can generate natural responses, while local fallbacks keep core study tools useful if the provider is unavailable.

Remember

Do not exaggerate. Say exactly what you built, explain why you built it that way and describe advanced improvements as future scope. That is how a beginner sounds confident and professional.